

This script computes statistics that are specific to germany, comparing the months Jan-Apr of the original reporting period (2012-2025) to the current year (2026)

Note, that most of this code is identical to that in [03_compute_drought_days.ipynb](#). Creating a diff between the two files should be the most convenient way to do fact checking for the first cells here.

```
import numpy as np
import re
import matplotlib.pyplot as plt
import geopandas as gpd
import pandas as pd
import rasterio
import glob
import os
import utils
```

```
from tqdm.notebook import tqdm
```

```
tqdm.pandas()
```

```
cdi_files = sorted(
    glob.glob(
        os.path.join(
            utils.raw_data_dir, "combined_drought_indicator_2012-2025", "**", "*.tif"
        )
    )
)
cdi_folders = sorted(
```

```

    glob.glob(
        os.path.join(utils.raw_data_dir, "combined_drought_indicator_2012-2025", "*")
    )
)

# like before, toggle this variable to switch between LAU and NUTS-3 level of analysis
LAU_level = False
if LAU_level:
    raster_input_dir = "cdi_raster_coverage_lau"
else:
    raster_input_dir = "cdi_raster_coverage_nuts3"

# load only the country-file for germany into a single DataFrame
df = gpd.read_file(os.path.join(utils.intermediate_data_dir, raster_input_dir, "DE.geojson"))
lau_crs = df.crs

# load the first cdi file to extract metadata
with rasterio.open(cdi_files[0]) as src:
    geotiff = src.read(1)
    cdi_dims = src.read(1).shape

    assert src.crs == lau_crs

    # compute pixel indices in the raster for each pair of x/y coordinates
    min_coords = np.array(
        [src.index(x, y) for x, y in df[["xmin", "ymin"]].values.astype(float)]
    )
    max_coords = np.array(
        [src.index(x, y) for x, y in df[["xmax", "ymax"]].values.astype(float)]
    )

# enter pixel indices into the dataframe for later reference
df["row_start"] = min_coords[:, 0]
df["col_start"] = min_coords[:, 1]
df["row_end"] = max_coords[:, 0]
df["col_end"] = max_coords[:, 1]

# minor type cleanup
for col in ["col_start", "col_end", "row_start", "row_end"]:
    df[col] = df[col].astype(int)
df["cell_coverage"] = df["cell_coverage"].apply(lambda x: np.array(eval(x)))

```

```

def calculate_cell_weights(cell_coverage):
    # by default, exactextract returns the fraction of each grid cell that is being covered by the
    # We however desire the inverse: How much of the Polygon's area lies in a given grid cell
    # The latter allows us to weigh grid values to determine the average value inside the LAU
    total_coverage = cell_coverage.sum(axis=None)
    return cell_coverage / total_coverage

df["cell_weights"] = df["cell_coverage"].apply(calculate_cell_weights)

```

```

# compute two 3D-matrices containing the total number of "warning" and "alert" days per pixel
warning_days = np.zeros(((2027 - 2012), cdi_dims[0], cdi_dims[1]))
alert_days = np.zeros(((2027 - 2012), cdi_dims[0], cdi_dims[1]))

for i, year in enumerate(range(2012, 2027)):
    print("processing year: ", year)

    year_folder = cdi_folders[i]
    year_files = sorted(glob.glob(os.path.join(year_folder, "*.tif")))
    yearly_data = np.zeros(
        shape=(12, cdi_dims[0], cdi_dims[1]), dtype=np.uint8
    )

    for file_idx, path in enumerate(year_files):
        # NOTE: this is the important bit: in this script we only compare the first four months
        if file_idx > 11:
            break
        with rasterio.open(path) as src:
            yearly_data[file_idx] = src.read(1)

    # value 2 corresponds to CDI level "Warning", value 3 corresponds to level "Alert".
    # See page 3 https://drought.emergency.copernicus.eu/data/factsheets/factsheet\_combinedD
    cur_year_warning = yearly_data == 2
    warning_days[i] = cur_year_warning.sum(axis=0) * 10

    cur_year_alert = yearly_data == 3
    alert_days[i] = cur_year_alert.sum(axis=0) * 10

```

```

processing year: 2012
processing year: 2013

```

```

processing year: 2014
processing year: 2015
processing year: 2016
processing year: 2017
processing year: 2018
processing year: 2019
processing year: 2020
processing year: 2021
processing year: 2022
processing year: 2023
processing year: 2024
processing year: 2025
processing year: 2026

```

```

def calculate_yearly_stats(row):
    """generates three sets of arrays for each row in the dataframe: "warning days", "alert days", and "cell weights"

    Args:
        row (pd.Series): a row in the dataframe, representing either a LAU or NUTS-3

    Returns:
        pd.Series: the same row, including three new columns
    """
    cell_weights = row["cell_weights"]
    width, height = cell_weights.shape

    # NOTE: @factcheck: I messed this up previously. I believe it to be right like this, but
    x0, x1 = row["row_start"] - width, row["row_start"]
    y0, y1 = row["col_start"], row["col_start"] + height

    # take the subset of the 3D-arrays that overlaps with the shape of this region
    # then, multiply each value by the amount of the total area it comprises
    # finally, summ over axes 2 and 3 to get a 1D-array representing the "warning" and "alert"
    warning_days_per_year = (
        (warning_days[:, x0:x1, y0:y1] * cell_weights).sum(axis=(1, 2)).round(2)
    )
    alert_days_per_year = (
        (alert_days[:, x0:x1, y0:y1] * cell_weights).sum(axis=(1, 2)).round(2)
    )

    row["warning_days"] = warning_days_per_year.tolist()
    row["alert_days"] = alert_days_per_year.tolist()

```

```

        row["drought_days"] = (
            (warning_days_per_year + alert_days_per_year).round(2).tolist()
        )
    return row

print("calculating yearly stats...")
df = df.progress_apply(calculate_yearly_stats, axis=1)

```

calculating yearly stats...

```
0%|          | 0/401 [00:00<?, ?it/s]
```

```

# compute median value, maximum value and the year that the maximum value occurred for each
# NOTE: all secondary metrics are only computed up until 2025, so we can compare the 2026 va
for value in ["warning", "alert", "drought"]:
    df[f"median_{value}_days"] = df[f"{value}_days"].apply(
        lambda x: np.median(x[:-1]).round(2)
    )
    df[f"max_{value}_days"] = df[f"{value}_days"].apply(lambda x: np.max(x[:-1]).round(2))
    df[f"max_{value}_days_year"] = df[f"{value}_days"].apply(
        lambda x: 2012 + np.argmax(x[:-1])
    )
    df[f"2026_{value}_days"] = df[f"{value}_days"].apply(lambda x: x[-1])
    df[f"2026_{value}_days"] = df[f"{value}_days"].apply(lambda x: x[-1])

```

```

# drop columns that are no longer needed
if LAU_level:
    out_data = gpd.GeoDataFrame(
        df[
            [
                "CNTR_CODE",
                "LAU_NAME",
                "geometry",
                "warning_days",
                "median_warning_days",
                "max_warning_days",
                "max_warning_days_year",
                "alert_days",
                "median_alert_days",
                "max_alert_days",
            ]
        ]
    )

```

```

        "max_alert_days_year",
        "drought_days",
        "median_drought_days",
        "max_drought_days",
        "max_drought_days_year",
    ]
]
)
else:
    out_data = gpd.GeoDataFrame(
        df[
            [
                "NUTS_ID",
                "CNTR_CODE",
                "NAME_LATN",
                "NUTS_NAME",
                "geometry",
                "warning_days",
                "median_warning_days",
                "max_warning_days",
                "max_warning_days_year",
                "alert_days",
                "median_alert_days",
                "max_alert_days",
                "max_alert_days_year",
                "drought_days",
                "median_drought_days",
                "max_drought_days",
                "max_drought_days_year",
            ]
        ]
    )

out_data = out_data.to_crs(epsg=4326)

```

```

df = df.to_crs(epsg=3035) # convert to a metric CRS to compute surface area
df["area_km2"] = round(df.geometry.area / 1e6, 2)

```

```

df[df["max_drought_days"] < df["2026_drought_days"]]
# in this cell we select all rows where the value for 2026 is higher than the maximum of all

```

	NUTS_ID	LEVL_CODE	CNTR_CODE	NAME_LATN	NUTS_NAME	MOUNT_TYP
24	DE923	3	DE	Hameln-Pyrmont	Hameln-Pyrmont	4
164	DE732	3	DE	Fulda	Fulda	4
248	DE916	3	DE	Goslar	Goslar	3
339	DEA57	3	DE	Hochsauerlandkreis	Hochsauerlandkreis	2

→ **Hameln-Pyrmont, Fulda, Goslar und der Hochsauerlandkreis haben in den ersten vier Monaten des Jahres so viele DürreWARNtage erlebt, wie in keinem anderen ersten Jahresdrittel im Aufzeichnungszeitraum.**

```
df[df["max_alert_days"] < df["2026_alert_days"]]
# in this cell we select all rows where the number of alert days for 2026 is higher than the
```

	NUTS_ID	LEVL_CODE	CNTR_CODE	NAME_LATN	NUTS_NAME
105	DE21G	3	DE	Mühldorf a. Inn	Mühldorf a. Inn
223	DEE0D	3	DE	Stendal	Stendal
349	DEE01	3	DE	Dessau-Roßlau, Kreisfreie Stadt	Dessau-Roßlau, Kreisfreie Stadt

→ **Mühldorf a. Inn, Stendal und Dessau-Roßlau haben in den ersten vier Monaten des Jahres so viele DürreALARMtage erlebt, wie in keinem anderen ersten Jahresdrittel im Aufzeichnungszeitraum.**

```
mean_drought_days_baseline = (df["median_drought_days"]*df["area_km2"]).sum() / df["area_km2"].sum()
mean_drought_days_2026 = (df["2026_drought_days"]*df["area_km2"]).sum() / df["area_km2"].sum()
f"Mean warning days between 2012-2025: {mean_drought_days_baseline:.2f}. 2026 was {mean_drought_days_2026:.2f}"
```

'Mean warning days between 2012-2025: 14.47. 2026 was 27.33, marking an increase of 88.87 per cent'

```
mean_alert_days_baseline = (df["median_alert_days"]*df["area_km2"]).sum() / df["area_km2"].sum()
mean_alert_days_2026 = (df["2026_alert_days"]*df["area_km2"]).sum() / df["area_km2"].sum()
f"Mean alert days between 2012-2025: {mean_alert_days_baseline:.2f}. 2026 was {mean_alert_days_2026:.2f}"
```

'Mean alert days between 2012-2025: 0.08. 2026 was 0.51, marking an increase of 567.96 per cent'

—> das erste Drittel 2026 (Januar-April) war überdurchschnittlich trocken. Im Vergleich zu den vorherigen Jahren seit 2012 hat Deutschland in diesem Frühjahr über sechsmal so viele Tage erlebt, an denen die Dürre örtlich so schwer war, dass sie das Pflanzenwachstum beeinträchtigte.

```
# NUTS-2 code for Schleswig holstein is DEFO https://de.wikipedia.org/wiki/NUTS:DE  
df_schleswig_holstein = df[df["NUTS_ID"].apply(lambda x: x.startswith("DEFO"))]
```

```
mean_alert_days_baseline = (df_schleswig_holstein["median_warning_days"]*df_schleswig_holstein["area_sqkm"])/df_schleswig_holstein["area_sqkm"]  
mean_alert_days_2026 = (df_schleswig_holstein["2026_warning_days"]*df_schleswig_holstein["area_sqkm"])/df_schleswig_holstein["area_sqkm"]  
f"Mean alert days between 2012-2025: {mean_alert_days_baseline:.2f}. 2026 was {mean_alert_days_2026:.2f}, marking an increase of {((mean_alert_days_2026/mean_alert_days_baseline)-1)*100:.2f} per cent"
```

```
'Mean alert days between 2012-2025: 3.26. 2026 was 6.38, marking an increase of 96.00 per cent'
```

—> Schleswig Holstein erlebte ein ungewöhnlich trockenes erstes Jahresdrittel: hier wurden etwa zweimal so viele Dürrewarntage verzeichnet, wie in einem durchschnittsjahr zwischen 2012 und 2025.